第2次作業題目-作業-HW2

學號：113111149

姓名：王立杰

作業撰寫時間：約 43 mins（本次協作製作與驗證）

最後撰寫文件日期：2026/10/29

本份文件包含以下主題：

[x] 說明內容

[x] 其他

說明內容

本次於 Unity 6000.6.7f1、Input System 1.20.1 實作 RabbitController.cs。在獨立的 HW2 場景建立 Rabbit 物件並掛載腳本，原本的 QZ2 場景與 PlayerController.cs 保留。HW2 場景只有 RabbitController 負責輸出按鍵訊息。

a. 偵測四個單鍵

在 Update() 中取得 Keyboard.current，若沒有目前鍵盤便使用 return 結束本次更新。接著以四個獨立的 if 檢查 W、A、S、D，因此不同按鍵可以同時成立。

按鍵	讀取欄位	Console 訊息
W	keyboard.wKey.isPressed	玩家按了 W 鍵！
A	keyboard.aKey.isPressed	玩家按了 A 鍵！
S	keyboard.sKey.isPressed	玩家按了 S 鍵！
D	keyboard.dKey.isPressed	玩家按了 D 鍵！

原題 a 的 W 鍵欄位寫為 sKey；這裡依 W 鍵需求與 Topic 8 第 8.7 節採用 wKey，S 鍵才使用 sKey。isPressed 代表按住狀態，按住多幀會重複輸出。

b. 新增六組雙鍵

在 a 的四個判斷後，新增 W+A、W+S、W+D、A+S、A+D、S+D 六組判斷。每組以 && 連接兩個 isPressed，兩鍵都按住時才輸出該組合訊息，例如「玩家按了 W + A 鍵！」。

本次依「承上繼續新增」保留單鍵訊息，另外顯示組合訊息。按住 W+A 時會產生 W、A、W+A 三種訊息。所有條件使用獨立 if，沒有以 else if 排除其他成立的組合。

完整程式碼

Assets/Scripts/RabbitController.cs 最終版本共 64 行，包含空白行。以下程式碼與本機 Unity 中的已編譯腳本一致。

```
using UnityEngine;
using UnityEngine.InputSystem;

public class RabbitController : MonoBehaviour
{
    void Update()
    {
        Keyboard keyboard = Keyboard.current;
        if (keyboard == null)
        {
            return;
        }

        if (keyboard.wKey.isPressed)
        {
            Debug.Log("玩家按了 W 鍵！");
        }

        if (keyboard.aKey.isPressed)
        {
            Debug.Log("玩家按了 A 鍵！");
        }

        if (keyboard.sKey.isPressed)
        {
            Debug.Log("玩家按了 S 鍵！");
        }

        if (keyboard.dKey.isPressed)
        {
            Debug.Log("玩家按了 D 鍵！");
        }

        if (keyboard.wKey.isPressed && keyboard.aKey.isPressed)
        {
            Debug.Log("玩家按了 W + A 鍵！");
        }

        if (keyboard.wKey.isPressed && keyboard.sKey.isPressed)
        {
            Debug.Log("玩家按了 W + S 鍵！");
        }

        if (keyboard.wKey.isPressed && keyboard.dKey.isPressed)
        {
            Debug.Log("玩家按了 W + D 鍵！");
        }

        if (keyboard.aKey.isPressed && keyboard.sKey.isPressed)
        {
            Debug.Log("玩家按了 A + S 鍵！");
        }

        if (keyboard.aKey.isPressed && keyboard.dKey.isPressed)
        {
            Debug.Log("玩家按了 A + D 鍵！");
        }

        if (keyboard.sKey.isPressed && keyboard.dKey.isPressed)
        {
            Debug.Log("玩家按了 S + D 鍵！");
        }
    }
}
```

程式碼說明

行號	程式碼或區段	意義
1-2	using UnityEngine; 與 using UnityEngine.InputSystem;	匯入 MonoBehaviour、Debug 與鍵盤輸入所需的類別。
4	RabbitController : MonoBehaviour	宣告可掛載在 GameObject 上的公開腳本類別；檔名與類別名稱相同。
6	void Update()	在啟用的物件與元件上，由 Unity 每幀呼叫，不回傳資料。
8	Keyboard keyboard = Keyboard.current;	取得目前鍵盤參照並存入區域變數。
9-12	if (keyboard == null) 與 return;	沒有鍵盤時提早結束，避免後面讀取按鍵造成 NullReferenceException。
14-32	四個單鍵 if	檢查各按鍵的 isPressed，成立時輸出對應訊息；每個 if 都會獨立檢查。
34-37	W 與 A 的 && 判斷	W、A 同時被按住才輸出 W + A；只有其中一鍵時不輸出這條組合訊息。
39-62	其餘五組雙鍵 if	用同樣方式檢查 W+S、W+D、A+S、A+D、S+D。
各 Debug.Log 行	Debug.Log("玩家按了 … 鍵！");	將雙引號內的字串送到 Unity Console；分號結束敘述。
大括號	{ 與 }	分別界定類別、方法、條件成立時的敘述區塊；63、64 行依序結束方法與類別。

為什麼使用 &&

&& 是邏輯 AND，兩個條件都為 true 才成立。若改成 ||，只按其中一鍵就會成立，無法表達題目的雙鍵需求。

第一鍵按住	第二鍵按住	雙鍵條件結果
否	否	false，不輸出組合
是	否	false，不輸出組合
否	是	false，不輸出組合
是	是	true，輸出組合

若同時按三鍵，會有三種單鍵與三種雙鍵訊息；四鍵同時按住則有四種單鍵與六種雙鍵，共十種訊息。這是保留各個獨立 if 的結果。

其他

本次重點是區分按住狀態與單次按下事件，以及 && 和獨立 if 的用途。複合鍵成立時仍可保留單鍵功能，三鍵或四鍵也能輸出所有符合的組合。文件中的執行證據來自真實 Unity 與 VS Code 視窗；測試輸入使用 Input System 模擬事件，沒有將模擬測試說成人工按鍵。

測試方法與結果

第一階段：實際自動更新

先只完成 a，進入真正 Unity Play Mode，以 Input System 模擬 W、A、S、D 的按下與放開。測試先確認遊戲影格由 1 增加到 44，再逐一檢查四種訊息，全部通過。圖 1、圖 2 保存當時的程式碼與執行結果。

第二階段：16 種組合與邊界

加入 b 後，在 Play Mode 暫停自動更新，建立獨立的 Input System 測試鍵盤，逐次套用按鍵狀態，透過 SendMessage("Update") 呼叫真正的 RabbitController 方法。如此可檢查每一次 Update 的訊息集合，避免以不同長按時間造成的累計行數判斷正確性。

按住按鍵	單鍵訊息數	雙鍵訊息數	結果
未按鍵	0	0	通過
W	1	0	通過
A	1	0	通過
W + A	2	1	通過
S	1	0	通過
W + S	2	1	通過
A + S	2	1	通過
W + A + S	3	3	通過
D	1	0	通過
W + D	2	1	通過
A + D	2	1	通過
W + A + D	3	3	通過
S + D	2	1	通過
W + S + D	3	3	通過
A + S + D	3	3	通過
W + A + S + D	4	6	通過

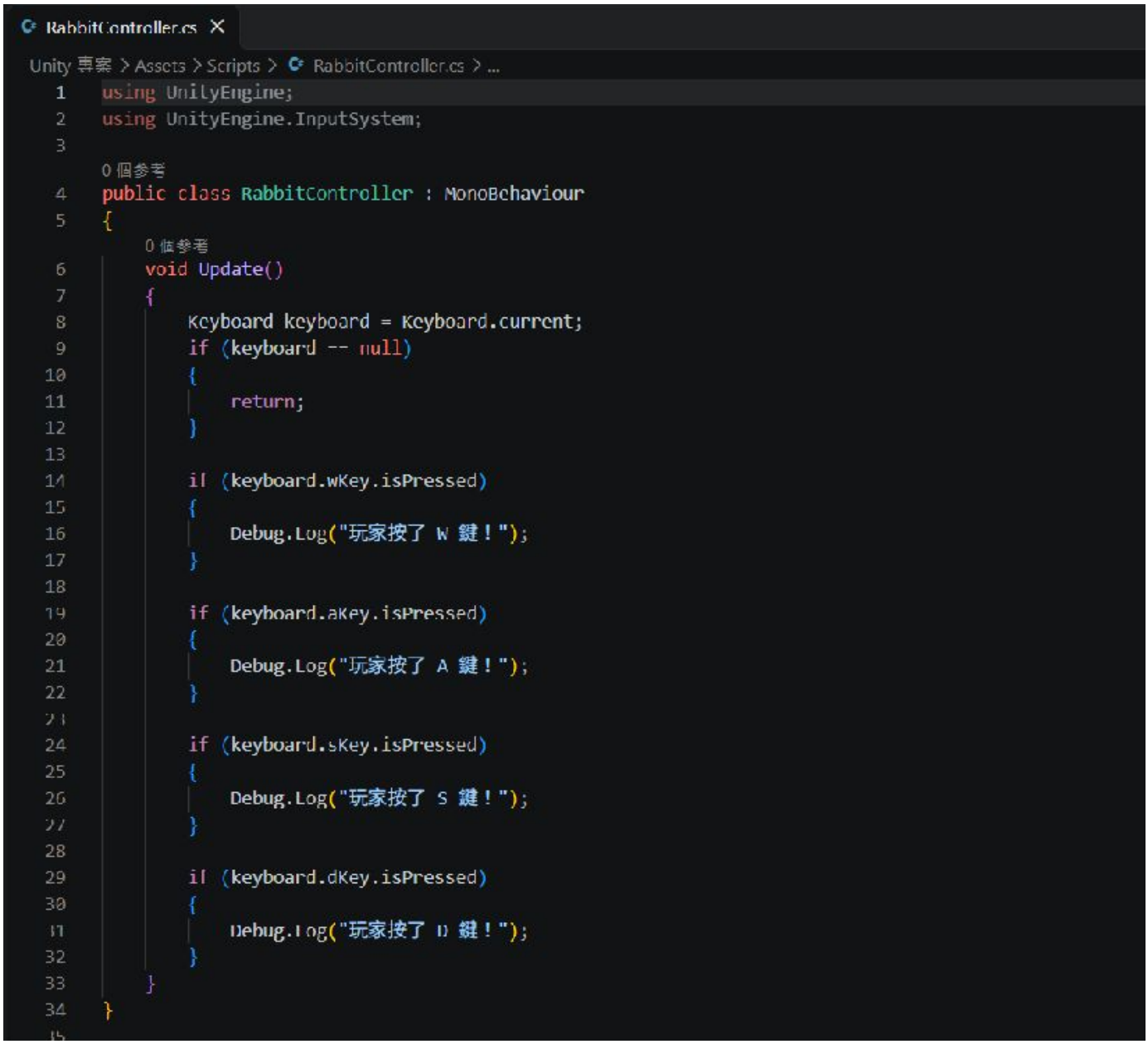
放開全部按鍵後沒有新訊息；同一個 W 按住狀態連續呼叫兩次 Update，各產生一條 W 訊息。空鍵盤防護則由測試工具短暫將 Keyboard.current 設為 null 後呼叫，再立即還原；沒有拔除實體鍵盤。三項邊界驗證皆通過，測試鍵盤已移除。

最終自動播放確認

恢復正常播放，再以 Input System 同時按住四鍵，確認遊戲影格有前進，實際 Console 出現四種單鍵與六種雙鍵共十種訊息。圖 4 的每種訊息均累計 35 次；Collapse 將相同內容合併。這些次數來自長按期間多次自動 Update，不代表只有一次按鍵事件。

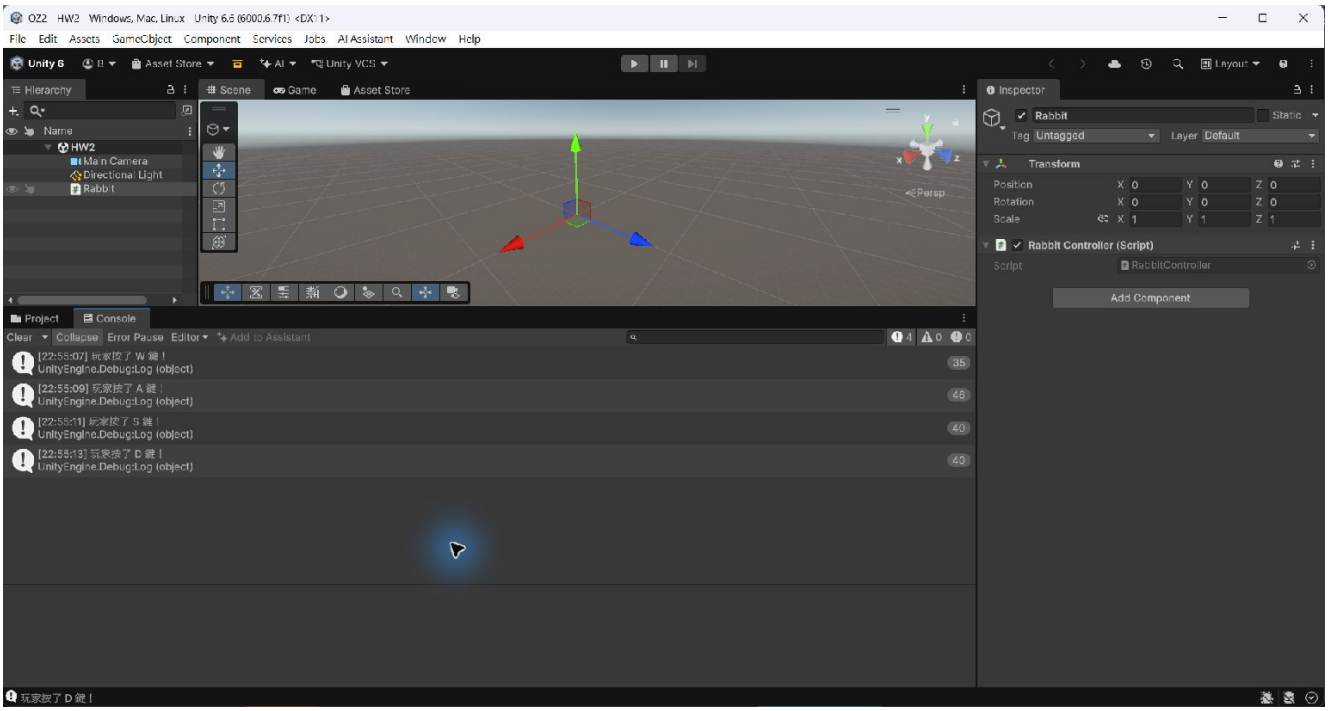
測試完成已停止播放。Unity Console 的警告與錯誤均為 0；HW2 場景保留 RabbitController 掛載狀態。

圖 1：a 的單鍵程式碼



VS Code 的第一階段原始畫面，四個單鍵判斷使用 wKey、aKey、sKey、dKey。PDF 以同一張原圖的程式碼區域呈現，原圖完整保留於 screenshots。

圖 2：a 的 Unity 執行結果

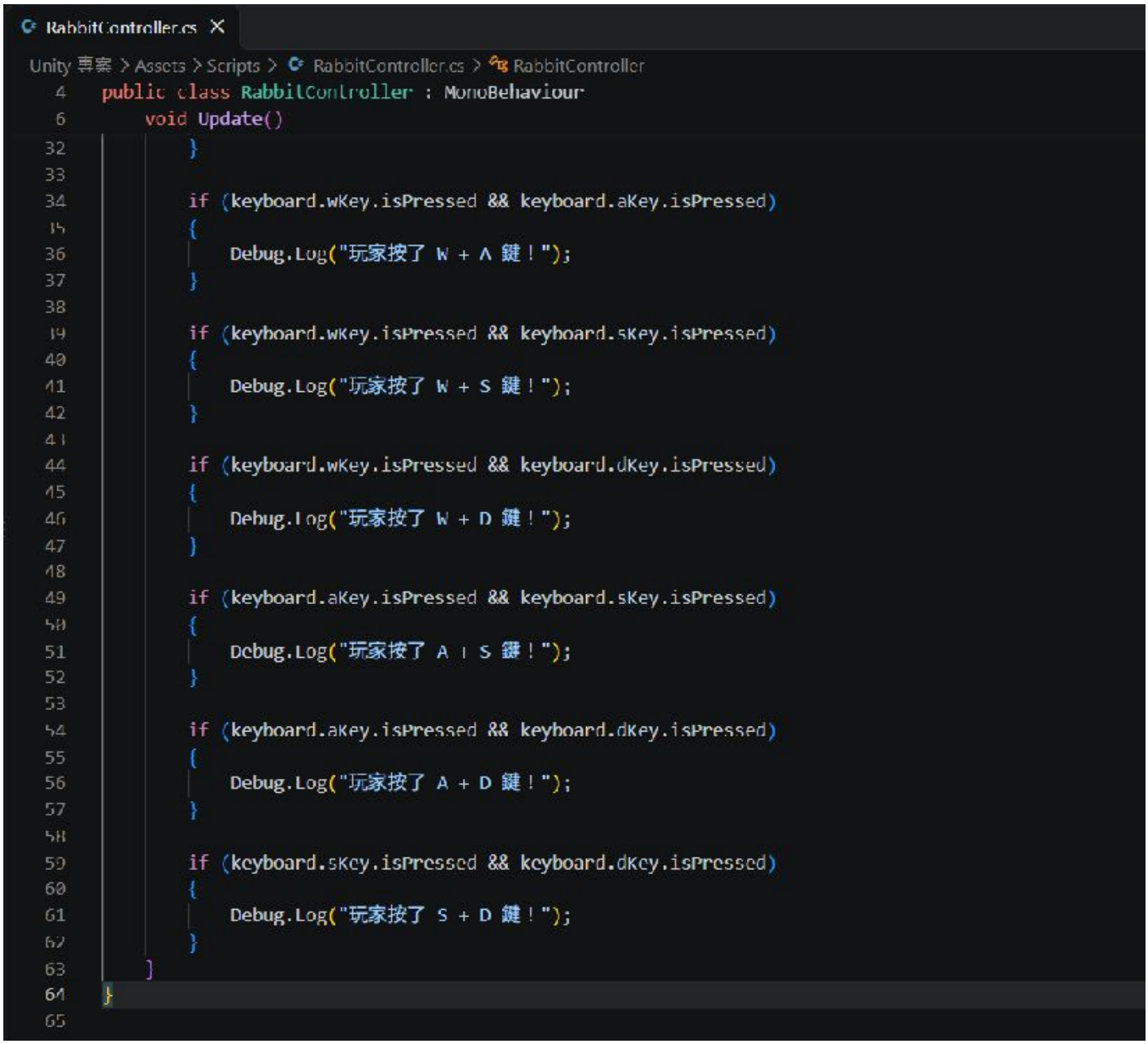


Console 訊息局部放大（同一張原始截圖）



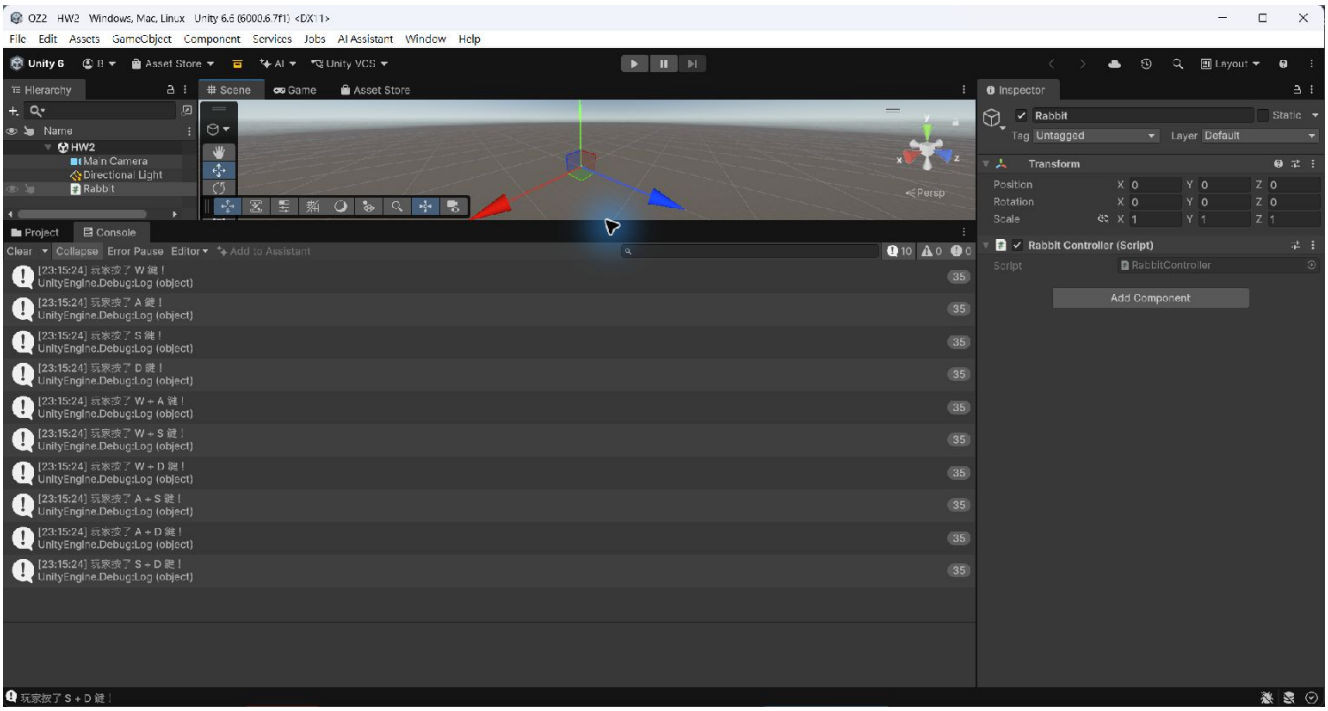
HW2 場景中的 Rabbit 已掛載 RabbitController；Console 顯示 W、A、S、D 四種訊息，累計次數依序為 35、46、40、40，警告與錯誤均為 0。放大區域取自同一張原始截圖。

圖 3：b 的六組複合鍵程式碼



最終程式第 34-62 行依序呈現六組 && 判斷；每一組使用獨立 if，符合時輸出該組合文字。PDF 使用同一張原圖的程式碼區域，以利閱讀。

圖 4：最終十種訊息與腳本掛載



Console 訊息局部放大（同一張原始截圖）



正常 Play Mode 自動 Update 的最後驗證結果：四種單鍵加上 W+A、W+S、W+D、A+S、A+D、S+D 六種組合全部出現，RabbitController 仍掛載於 Rabbit。右上角 Console 計數為十種 Log、零警告、零錯誤。放大區域取自同一張原始截圖。